

OOP

UNIT 10.5 – ITERATORS: A GENERIC WAY TO TRAVERSE CONTAINERS

A container stores the elements. An iterator gives us a common way to reach them.

FROM GENERIC TYPES TO GENERIC TRAVERSAL

Unit 10 introduced generic programming. A function template can express one algorithm for many element types, and a class template can express one class design for many stored types. The next question is different: how can one algorithm move through different containers without being rewritten for each container?

So far, most of our loops have depended on indexes. That works naturally for arrays and for our `Collection<T>` class because both expose elements in contiguous positions. But not every container is designed around numeric indexing. If generic programming is meant to separate algorithms from concrete types, traversal should also be separated from one concrete container representation.

THE TRAVERSAL PROBLEM

Suppose we want to print every element in an array. The familiar solution uses an index:

```
int values[] = { 10, 20, 30, 40 };  
  
for (int i = 0; i < 4; ++i)  
{  
    cout << values[i] << endl;  
}
```

The loop works because an array supports direct indexing with []. A vector also supports indexing, so a similar loop is possible there. However, generic library code should not assume that every useful container provides operator[] or that every container stores elements in a way that makes numeric indexing the natural traversal mechanism.

We need a more general idea: an object that represents a position in a sequence and can move from one element to the next. In C++, that idea is an iterator.

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain why iterators separate traversal from a particular container representation.
- Describe an iterator as an object that refers to a position in a sequence.
- Use `begin()` and `end()` to traverse a container.
- Explain why `end()` represents the position one past the last element.
- Use dereference, increment, and comparison operations with iterators.
- Use explicit iterator types and `auto` appropriately.
- Use `const_iterator`, `cbegin()`, and `cend()` for read-only traversal.
- Relate range-based for loops to the `begin/end` iterator model.
- Write a simple generic function that operates on an iterator range.
- Explain how raw pointers can act as iterators for contiguous data.
- Add simple `begin()` and `end()` functions to `Collection<T>`.
- Recognize how STL algorithms such as `find` work on iterator ranges.

WHAT IS AN ITERATOR?

An iterator is an object that identifies a position in a sequence of elements. It provides a common way to access the current element and move through the sequence. The exact internal representation depends on the container, but code using the iterator does not need to know that representation.

A useful first mental model is a pointer. If a pointer refers to an array element, we can dereference it with `*` and advance it with `++`. Many iterators deliberately support similar syntax:

```
*it      // access the current element  
++it     // move to the next element
```

This similarity is intentional. Iterators generalize the idea of a pointer. A raw pointer can itself serve as an iterator in some situations, but not every iterator is literally a pointer.

THE THREE BASIC ITERATOR OPERATIONS

For the first iterator loops in this course, three operations matter most:

- Dereference: `*it` accesses the element at the current iterator position.
- Increment: `++it` moves the iterator to the next element.
- Comparison: `it != end` checks whether the traversal has reached the end position.

These operations are enough to express a simple forward traversal. Later we will see that some iterators support additional operations, but generic code should require only the capabilities it actually needs.

BEGIN() AND END()

Standard containers provide two fundamental functions for traversal. `begin()` returns an iterator referring to the first element. `end()` returns an iterator representing the position immediately after the last element.

```
auto first = values.begin();  
auto finish = values.end();
```

The end iterator does not refer to a valid element. It is a boundary marker. Therefore the usual loop condition is `it != values.end()`, and we must never dereference `values.end()`.

WHY END() IS ONE PAST THE LAST ELEMENT

The one-past-the-end convention may seem unusual at first, but it makes a range easy to describe. A sequence is represented by two positions: the first included position and the first excluded position.

```
[ begin, end )
```

This notation means begin is included and end is excluded. An empty range is especially simple: begin and end are equal. The same representation also allows algorithms to work on complete containers or on selected subranges without changing their basic loop structure.

The rule to remember is simple: `end()` is used for comparison, not for element access.

OUR FIRST ITERATOR LOOP

We will use `std::vector` only as a convenient container for demonstrating iterators. The next unit will study STL containers in more detail.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> values = { 10, 20, 30, 40 };
    for (vector<int>::iterator it = values.begin();
        it != values.end(); ++it) {
        cout << *it << endl;
    }
}
```

The loop starts with an iterator at the first element, repeatedly *dereferences* the current iterator, *advances* it, and *stops* when it reaches the *end* boundary.

```
10
20
30
40
```

READING VECTOR<INT>::ITERATOR

The type `vector<int>::iterator` should be read from the inside outward. `vector<int>` is the concrete container type. `::iterator` names the iterator type provided by that container.

The iterator type belongs to the container design. A `vector<int>::iterator` and a `vector<string>::iterator` are not interchangeable, just as `vector<int>` and `vector<string>` are different concrete types.

Writing the full type is useful while learning because it makes the relationship visible. Once that idea is clear, C++ can often infer the type for us.

USING AUTO WITH ITERATORS

Iterator type names can become long. Because `begin()` already determines the correct iterator type, `auto` is commonly used:

```
for (auto it = values.begin();  
    it != values.end();  
    ++it)  
{  
    cout << *it << endl;  
}
```

The meaning of the loop has not changed. `auto` does not make the iterator dynamically typed, and it does not postpone the type decision until run time. The compiler still determines one exact iterator type during compilation.

For this course, the important learning order is: first understand the explicit iterator type, then use `auto` to remove unnecessary type spelling.

ITERATORS CAN MODIFY ELEMENTS

When an iterator refers to a non-const container, dereferencing it may produce a modifiable element. For example:

```
vector<int> values = { 10, 20, 30 };  
  
for (auto it = values.begin(); it != values.end(); ++it)  
{  
    *it = *it + 1;  
}
```

After the loop, the vector contains 11, 21, and 31. The assignment modifies the actual elements stored in the container.

This is similar to modifying an array element through a pointer. If the algorithm is intended only to read values, the interface should express that intention with const traversal.

CONST ITERATORS

A const iterator prevents modification of the element through that iterator. One explicit form is `vector<int>::const_iterator`:

```
vector<int> values = { 10, 20, 30 };

for (vector<int>::const_iterator it = values.cbegin();
     it != values.cend();
     ++it)
{
    cout << *it << endl;
    // *it = 0;    // Error: element is read-only through this iterator
}
```

The functions `cbegin()` and `cend()` explicitly request const iterators. This is useful when an algorithm should observe a container without changing its elements.

The idea is the same const-correctness principle used throughout the course: when an operation does not need to modify an object, its interface should not permit modification accidentally.

A CONST CONTAINER PRODUCES READ-ONLY TRAVERSAL

If the container itself is const, ordinary begin() also produces a read-only iterator type:

```
const vector<int> values = { 10, 20, 30 };

for (auto it = values.begin(); it != values.end(); ++it)
{
    cout << *it << endl;
    // *it = 5;    // Error
}
```

This mirrors const member-function behavior from earlier units. A const object exposes only operations that preserve its logical state, and traversal through it must not provide a hidden path to modify its elements.

RANGE-BASED FOR: A SIMPLER TRAVERSAL SYNTAX

C++ provides a range-based for loop for the common case in which we simply want to visit every element:

```
vector<int> values = { 10, 20, 30 };

for (const int& value : values)
{
    cout << value << endl;
}
```

This syntax is shorter, but conceptually it relies on the same range idea: determine the beginning, determine the end, and move through the elements between them.

Using `const int&` avoids copying each element and prevents modification. For a `vector<Song>`, the same pattern becomes especially useful because `Song` objects may be more expensive to copy than integers.

```
for (const Song& song : songs)
{
    song.print();
}
```

WHEN SHOULD WE USE AN EXPLICIT ITERATOR LOOP?

Range-based `for` is excellent when the goal is simply to visit every element. An explicit iterator is useful when the algorithm needs the iterator itself—for example, when calling an STL algorithm, comparing a found position with `end()`, working with a subrange, or performing an operation whose result is another iterator.

The shorter syntax does not replace iterators. It provides a convenient interface for one very common iterator-based traversal pattern.

FROM TYPE-GENERIC TO CONTAINER-GENERIC CODE

Unit 10 showed that a template can make an algorithm independent of one concrete element type. Iterators let us take another step: the algorithm can also depend less on the concrete container.

Consider a generic printing function that receives two iterator positions:

```
template <typename Iterator>
void printRange(Iterator begin, Iterator end)
{
    while (begin != end)
    {
        cout << *begin << endl;
        ++begin;
    }
}
```

The template does not mention vector, array, Collection, int, or Song. It describes only the operations required from Iterator: comparison with another iterator, dereference, and increment.

USING PRINTRANGE WITH A VECTOR

```
vector<int> values = { 10, 20, 30 };  
  
printRange(values.begin(), values.end());
```

The compiler deduces `Iterator` from the iterator arguments. The function traverses the vector without knowing that the container is a vector.

RAW POINTERS CAN BE ITERATORS

A raw pointer already supports the three operations used by `printRange`: comparison, dereference, and increment. Therefore pointers can act as iterators for an array:

```
int values[] = { 10, 20, 30, 40 };  
  
printRange(values, values + 4);
```

The first argument points to the first array element. `values + 4` points one position past the last element. The same generic algorithm now works for a vector iterator range and for a raw-pointer range.

This example shows why iterators are often described as a generalization of pointers: algorithms can be written against pointer-like traversal operations rather than against one specific container.

ITERATOR REQUIREMENTS

The `printRange` template illustrates the same requirements-based thinking introduced in Unit 10. The template parameter name `Iterator` does not guarantee any capability by itself. The function body determines what the type must support.

- `begin != end` requires inequality comparison between iterator values.
- `*begin` requires dereference to access the current element.
- `++begin` requires forward movement to the next position.
- `cout << *begin` additionally requires the element itself to be printable.

If a supplied iterator type supports these operations, it can participate in this algorithm. Generic programming is therefore based on behavior required by the code, not on the spelling of a particular class name.

NOT ALL ITERATORS HAVE THE SAME CAPABILITIES

All useful iterators are not identical. Some support only forward movement with `++`. Others also support `--`. Iterators for random-access containers such as `vector` can support operations such as `it + 3` or `it[3]`.

A generic algorithm should not use stronger operations than it actually needs. Our `printRange` algorithm needs only forward traversal, so it is more reusable than a version that unnecessarily assumes random access.

The standard library gives formal names to iterator capability categories. We will not study that taxonomy in depth here. For this course, the important principle is that different iterator types provide different sets of operations, and an algorithm should be written according to the operations it truly requires.

ADDING ITERATORS TO COLLECTION<T>

Unit 10 introduced a teaching `Collection<T>` implemented with a dynamic array. Because its elements are stored contiguously, a simple pointer can serve as its iterator. We can expose `begin()` and `end()` directly:

```
template <typename T>
class Collection
{
private:
    T* items;
    int size;
    int capacity;

public:
    // Existing constructors and operations...

    T* begin();
    T* end();
    const T* begin() const;
    const T* end() const;
};
```

IMPLEMENTING BEGIN() AND END()

```
template <typename T> T* Collection<T>::begin() {  
    return items;  
}  
template <typename T> T* Collection<T>::end() {  
    return items + size;  
}  
template <typename T> const T* Collection<T>::begin() const {  
    return items;  
}  
template <typename T> const T* Collection<T>::end() const {  
    return items + size;  
}
```

The non-const overloads return `T*` so callers may modify elements. The const overloads return `const T*` so traversal through a `const Collection<T>` cannot modify the stored objects.

This is intentionally a simple teaching design. Standard-library containers define dedicated iterator types because their implementations and guarantees are more general than a raw dynamic array.

COLLECTION<T> NOW WORKS WITH ITERATOR-BASED CODE

```
Collection<int> numbers(5);  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
  
for (auto it = numbers.begin(); it != numbers.end(); ++it)  
{  
    cout << *it << endl;  
}
```

More importantly, the generic printRange function works without modification:

```
printRange(numbers.begin(), numbers.end());
```

The Collection class and the algorithm are now connected through a small traversal interface rather than through knowledge of each other's internal implementation.

RANGE-BASED FOR CAN USE BEGIN() AND END()

Once a type provides suitable `begin()` and `end()` operations, it can participate in range-style traversal. With the simple `Collection<T>` interface above, code can be written in the familiar form:

```
for (const int& value : numbers)
{
    cout << value << endl;
}
```

This is a useful design lesson. Small, conventional interfaces allow user-defined types to participate naturally in language and library patterns. We do not need to teach the loop anything special about `Collection<T>`; we make `Collection<T>` satisfy the traversal convention expected by the loop.

A FIRST STL ALGORITHM: FIND

The Standard Template Library contains *algorithms* that accept iterator ranges. The `find` algorithm searches from the first iterator up to, but not including, the end iterator.

```
#include <algorithm>
#include <vector>
vector<int> values = { 10, 20, 30, 40 };
auto it = find(values.begin(), values.end(), 30);
if (it != values.end())
    cout << "Found: " << *it << endl;
```

The return value is an iterator. If the target is found, the iterator refers to the matching element. If no match exists, `find` returns the same end iterator that marks the search boundary.

WHY FIND RETURNS AN ITERATOR

Returning an iterator is more useful than returning only `true` or `false`. The caller can test whether a match exists and, when it does, use the returned position to inspect or modify the element according to the container and iterator type.

The algorithm also avoids depending on a numeric index. A numeric index would make sense for `vector` but not for every possible container. Returning an iterator preserves the generic design.

FIND AND USER-DEFINED TYPES

The find algorithm compares elements with the requested value. Therefore a user-defined type such as Song can participate when the required equality operation is available. If Song defines a meaningful operator==, the same algorithm can search a vector<Song> or another compatible iterator range.

This reconnects three ideas from earlier units: operator overloading defines meaningful class operations, templates express reusable algorithms, and iterators describe the range on which those algorithms operate.

ANOTHER EXAMPLE: COUNT

The count algorithm uses the same range representation and counts values equal to a target:

```
#include <algorithm>
#include <vector>

vector<int> values = { 10, 20, 10, 30, 10 };

int howMany = count(values.begin(), values.end(), 10);

cout << howMany << endl;    // 3
```

The important point is not to memorize find and count. It is to recognize their design: both receive a pair of iterators describing a range. The algorithm is separated from the concrete container.

ITERATOR VALIDITY

An iterator is meaningful only while the position it represents remains valid. Some container operations may move elements or rebuild internal storage, which can invalidate existing iterators. For example, growing a vector may require allocating a new memory block and moving its elements.

```
auto it = values.begin();  
  
values.push_back(50);    // May change the vector  
  
// Do not assume that the old it is still valid.
```

The exact invalidation rules depend on the container and the operation. We will not memorize those rules in this unit. The safe principle is to avoid keeping iterators across structural modifications unless the container documentation guarantees that they remain valid.

PAUSE AND PREDICT

Assume `vector<int> values = { 5, 10, 15 };`. Decide what each expression means and whether it is valid:

- `values.begin()`
- `*values.begin()`
- `values.end()`
- `*values.end()`
- `++it` when it refers to the first element
- `it != values.end()`

The key distinction is between an iterator position and the element at that position. `begin()` and `end()` produce iterator positions. Dereference accesses an element only when the iterator actually refers to one.

GUIDED PRACTICE: PRINTTITLES

Assume Song provides getTitle() const. Write a function template that receives two iterators and prints the title of every Song-like element in the range. The algorithm should know nothing about the concrete container.

```
template <typename Iterator>
void printTitles(Iterator begin, Iterator end)
{
    while (begin != end)
    {
        cout << begin->getTitle() << endl;
        ++begin;
    }
}
```

The -> operator is convenient when an iterator dereferences to an object. The expression begin->getTitle() is equivalent in meaning to (*begin).getTitle(). This template requires the iterator to support comparison, increment, and member access to an element that provides getTitle().

INDEPENDENT CHECK: DESIGN BEFORE CODING

A media application needs a generic operation that receives a range and reports how many elements satisfy a simple comparison with a target value. Before writing the implementation, answer these questions:

1. Why should the operation receive two iterator positions instead of a vector parameter?
2. Which iterator operations are required if the range is traversed only forward?
3. Which element operation is required if each element is compared with target using `==`?
4. Why should target usually be passed as `const T&` rather than by value?
5. How could the same algorithm work with `vector<Song>` after `Song` defines `operator==`?

DESIGN GUIDELINES

- Think of an iterator as a position in a sequence, not as the element itself.
- Treat `end()` as a boundary marker and never dereference it.
- Use range-based for when you simply need to visit every element.
- Use explicit iterators when you need positions, subranges, or iterator-returning algorithms.
- Prefer `const` traversal when an algorithm only reads elements.
- Write generic algorithms against the weakest iterator operations they really need.
- Remember that raw pointers can serve as iterators for contiguous arrays.
- Expose `begin()` and `end()` when a user-defined collection can provide a meaningful traversal range.
- Assume iterators may become invalid after structural container changes unless the container guarantees otherwise.
- Use standard-library algorithms when they already express the operation clearly instead of rewriting the loop.

COMMON BEGINNER MISTAKES

- Dereferencing `end()` as if it referred to the final element.
- Confusing an iterator with the element reached through the iterator.
- Forgetting to increment the iterator and creating an infinite loop.
- Using `<=` or `<` as a general iterator loop condition instead of the portable `!= end` pattern.
- Assuming every iterator supports random-access expressions such as `it + 5`.
- Using a modifiable iterator when the algorithm should only read elements.
- Keeping an iterator after a container operation that may have invalidated it.
- Thinking `auto` makes iterator typing a run-time decision.
- Assuming range-based `for` is unrelated to iterators.
- Writing a generic algorithm for vector specifically when an iterator range would make it reusable.

SUMMARY

Iterators provide a common way to describe positions and ranges inside containers. Instead of requiring every algorithm to know how a particular container stores its elements, an iterator-based algorithm can work through a small set of traversal operations such as dereference, increment, and comparison.

The `begin()/end()` convention represents a range as `[begin, end)`: the first position is included and the end position is excluded. This representation works naturally for empty ranges, complete containers, and subranges. The end iterator is a boundary and must never be dereferenced.

Iterators connect directly to generic programming. A function template can accept an iterator type and operate on arrays, vectors, or user-defined collections as long as the supplied iterators satisfy the required operations. Raw pointers can act as iterators, and our `Collection<T>` can expose pointer-based `begin()` and `end()` functions because its data is stored contiguously.

STL algorithms such as `find` and `count` use iterator ranges so that the algorithms remain independent of one specific container. This is the larger design idea: templates make code generic over types, while iterators help make algorithms generic over traversal and container representation.

REVIEW QUESTIONS

1. What problem do iterators solve that numeric indexing does not solve generically?
2. What does an iterator represent?
3. Why are iterators often compared to pointers?
4. What do dereference, increment, and iterator comparison mean?
5. What does `begin()` return?
6. Why does `end()` represent one position past the last element?
7. Why must `end()` never be dereferenced?
8. What does `vector<int>::iterator` mean?
9. Why is `auto` commonly used for iterator variables?
10. How does a `const_iterator` differ from a modifiable iterator?
11. What is the relationship between `cbegin()/cend()` and const-correctness?
12. How is a range-based for loop related conceptually to `begin()` and `end()`?
13. Which operations does the `printRange` template require from its `Iterator` type?
14. Why can a raw pointer be used as an iterator for an array?

15. How can `Collection<T>` provide simple `begin()` and `end()` functions?
16. Why does `find` return an iterator rather than an integer index?
17. What does `find` return when the target is not present?
18. Why can not every iterator support expressions such as `it + 3`?
19. What is iterator invalidation?
20. How do templates and iterators work together to support STL algorithms?

EXIT CHECK

Complete these four sentences without looking back:

1. An iterator represents
2. The range `[begin, end)` includes ... but does not include
3. A generic iterator algorithm works when its iterator type supports
4. Templates make an algorithm independent of ..., while iterators help make it independent of

LOOKING AHEAD

We now have the two ideas needed to understand the architecture of the Standard Template Library. Templates let one container or algorithm design work with many types. Iterators provide a common language for describing ranges inside different containers.

In the next unit, we will use these ideas rather than rebuilding them. We will work with selected STL containers such as vector, traverse them using iterators and range-based loops, and apply standard algorithms to iterator ranges. The goal is not to memorize a catalog of library functions. The goal is to recognize how the generic-programming principles from Unit 10 and the traversal principles from this unit combine in professional C++ library design.